UNIVERSIDAD NACIONAL DEL CENTRO DEL PERÚ
FACULTAD DE INGENIERÍA DE SISTEMAS
DEPARTAMENTO ACADÉMICO DE INGENIERÍA DE SISTEMAS PROGRAMA DE INGENIERÍA DE SISTEMAS

GUÍA PRÁCTICA SEMANA 05

Asignatura: Desarrollo de Aplicaciones Web (IS093A)
Unidad I: Desarrollo Web Frontend
Tema: Framework JS (React), Client-Side Rendering (CSR), JSX, Props & Children, Estrategias de Estilado
Duración: 90 minutos

APELLIDOS Y NOMBRES: PARIONA RAMOS SAUL ANIBAL

OBJETIVO DE LA PRÁCTICA

Crear una aplicación frontend de página única (SPA) utilizando un framework JS (React), configurar el entorno con herramientas modernas (Vite + npm/pnpm), implementar arquitectura basada en componentes con JSX, transmitir datos mediante props y composición con children, aplicar estrategias de estilado (CSS Modules / Tailwind / inline) y validar el comportamiento de **Client-Side Rendering (CSR)** frente al ciclo de vida del DOM.

RECURSOS REQUERIDOS

- Node.js LTS (≥20.x) + npm/pnpm
- Visual Studio Code (Extensiones: ES7+ React/Redux snippets, Tailwind CSS IntelliSense, Error Lens, React Developer Tools)
- Navegador moderno + DevTools + extensión React DevTools
- Terminal integrada
- Documentación oficial: react.dev, vitejs.dev
- Conexión a internet

PARTE 1: EJERCICIO PASO A PASO

Paso	Actividad	Tiempo	Concepto Clave	Restricción/Nota Técnica
1	Scaffold del proyecto: npm create vite@latest, seleccionar React + JavaScript, instalar dependencias,	7 min	Gestión de dependencias, bundler (Vite), HMR, estructura de carpetas	No usar createreact-app (obsoleto). Verificar que package.json tenga

	ejecutar npm run dev			@vitejs/pluginreact
2	Limpiar boilerplate, crear src/components/, src/styles/, configurar alias o rutas relativas. Estructurar App.jsx como orquestador	8 min	Arquitectura de componentes, árbol de render, JSX → React.createElement	Mantener un solo componente raíz <App> que importa hijos. Evitar lógica de negocio en App
3	Implementar patrón props y children: crear <Dashboard> y <MetricCard>. Pasar datos estructurados y contenido anidado	12 min	Flujo unidireccional, inmutabilidad de props, composición vs herencia, PropTypes/JSDoc	Documentar tipos con JSDoc /** @param {Object} props */. No usar state en este paso
4	Aplicar estrategias de estilado: CSS Modules para <MetricCard>, utilidades Tailwind (si se configura) o inline para contrastar scope y colisiones	10 min	Scope de estilos, className vs class, CSS Modules hashing, prioridad de cascada en CSR	Prohibido usar <style> global en componentes. Usar *.module.css explícito

5	Validar CSR: inspeccionar HTML inicial (View Page Source), verificar hidratación en React DevTools, analizar red (JS	8 min	CSR vs SSR, hidratación, bundle analysis, index.html vacío vs DOM inyectado	Explicar por qué meta viewport y SEO tags deben ir en index.html, no en JSX
	chunks) y ciclo de render			

USO DE HERRAMIENTAS DE IA (Recomendado para optimizar el ejercicio)

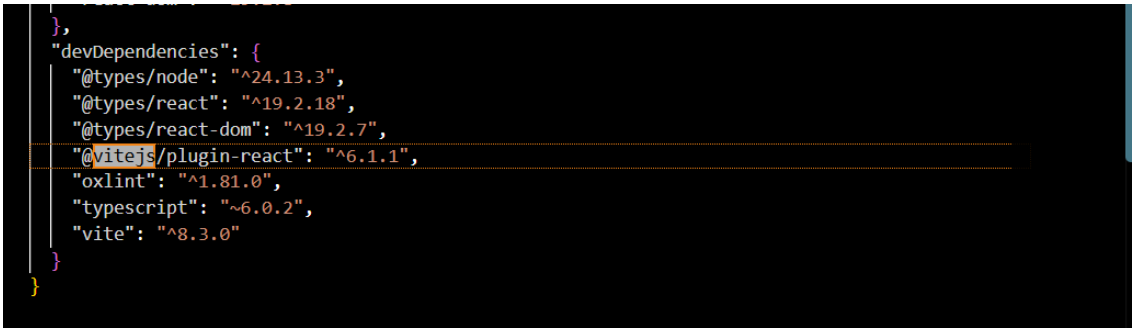
Herramienta	**Uso Permitido**	**Prompt Ejemplo**	**Uso Prohibido**
ChatGPT/Claude /Copilot	Explicar diferencias props vs children, depurar errores de hidratación, generar vite.config.js base	"¿Por qué mi componente hijo no recibechildrensi usoprops.childrenexplícitamente?"	"Genera la app completa de dashboard con React y Tailwind"
React DevTools	Inspeccionar árbol de componentes, verificar props pasadas, medir rerenders	(Extensión oficial)	Modificar estado directamente desde DevTools para "arreglar" la UI
Bundle Analyzer / Lighthouse	Medir tamaño de chunks JS, impacto de CSR en FCP/LCP	(Automático)	Ignorar advertencias de Missing key prop in list o Invalid prop type

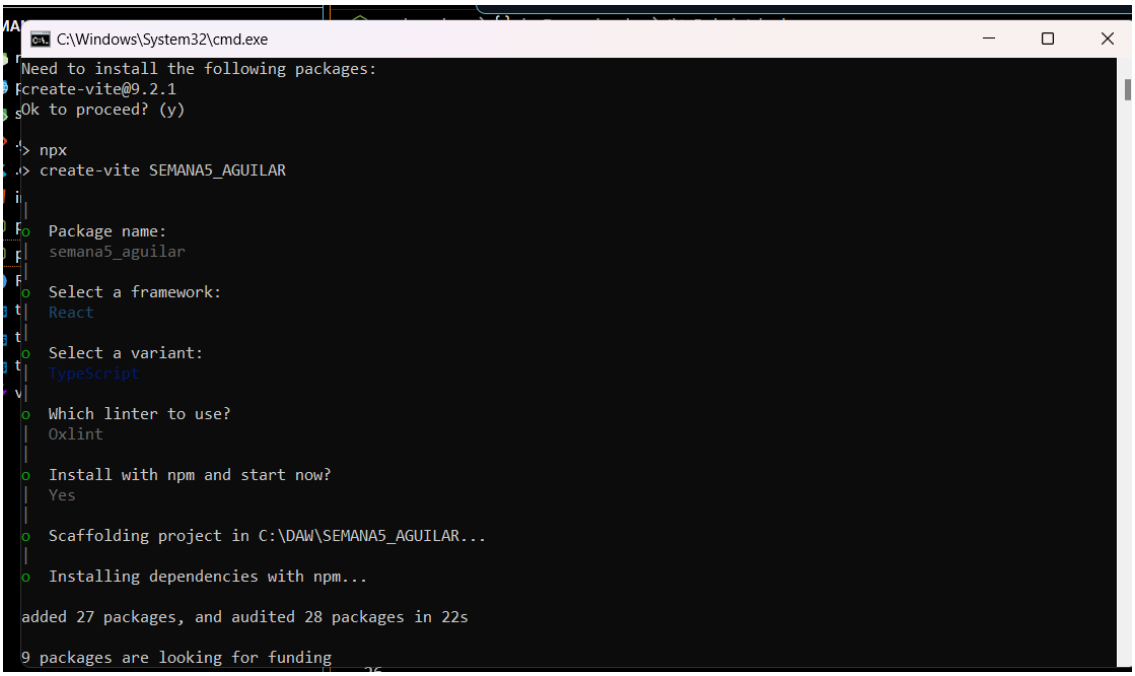
Regla de laboratorio: El 75% de la estructura de componentes y transmisión de datos debe escribirse manualmente. Cada uso de IA para resolver conflictos de estilos o flujo de props debe llevar comentario // IA: [problema] → Solución manual: [explicación]

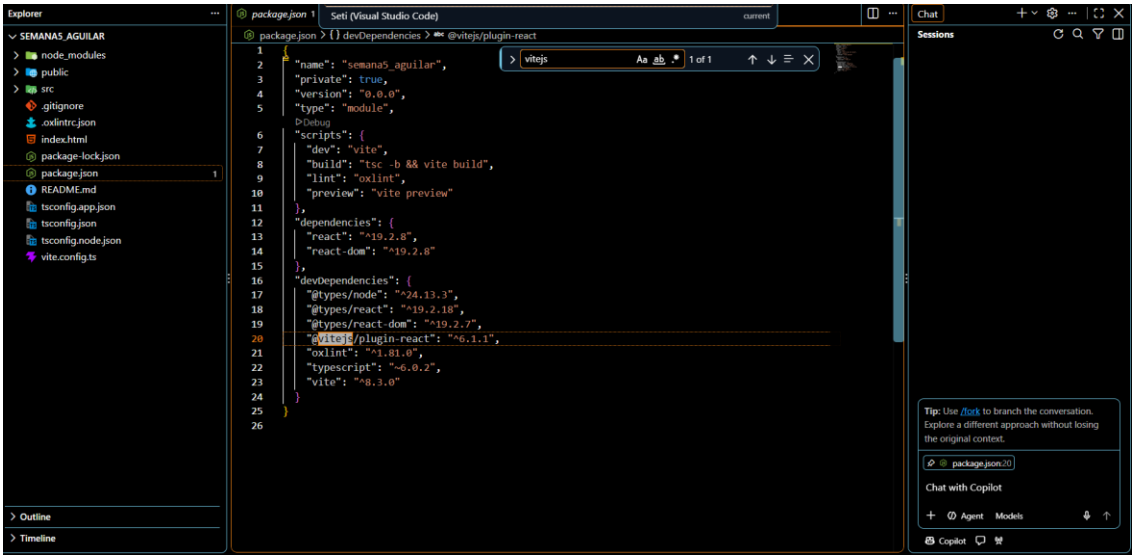
Evidencias del Ejercicio Desarrollado:

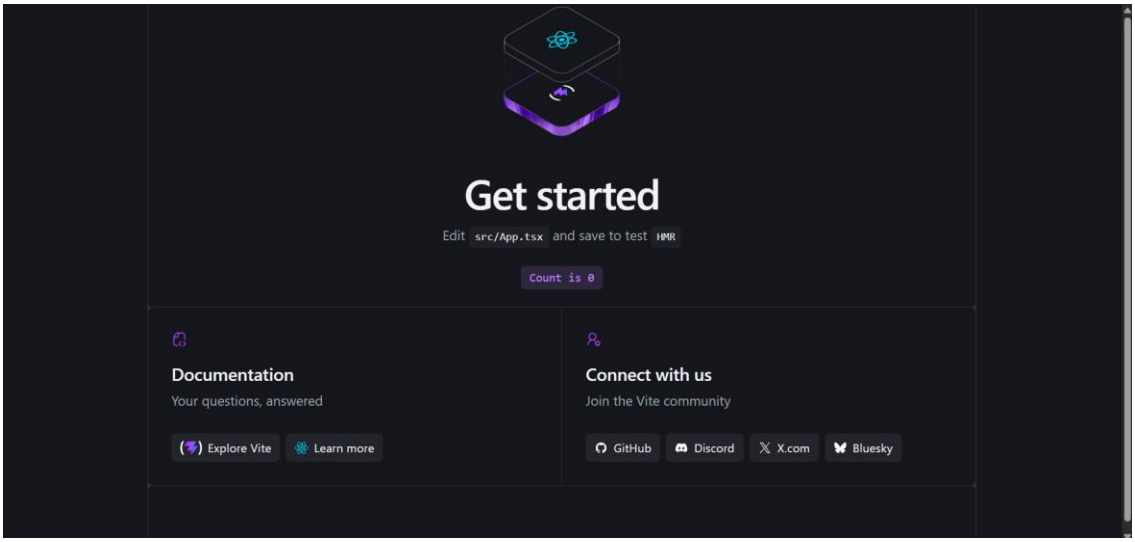
- Añada capturas de pantalla de cada paso realizado en Visual Studio Code.

Paso	Actividad	Tiempo	Concepto Clave	Restricción/Nota Técnica
1	Scaffold del proyecto: npm create vite@latest, seleccionar React + JavaScript, instalar dependencias, ejecutar npm run dev	7 min	Gestión de dependencias, bundler (Vite), HMR, estructura de carpetas	No usar createreact-app (obsoleto). Verificar que package.json tenga @vitejs/pluginreact

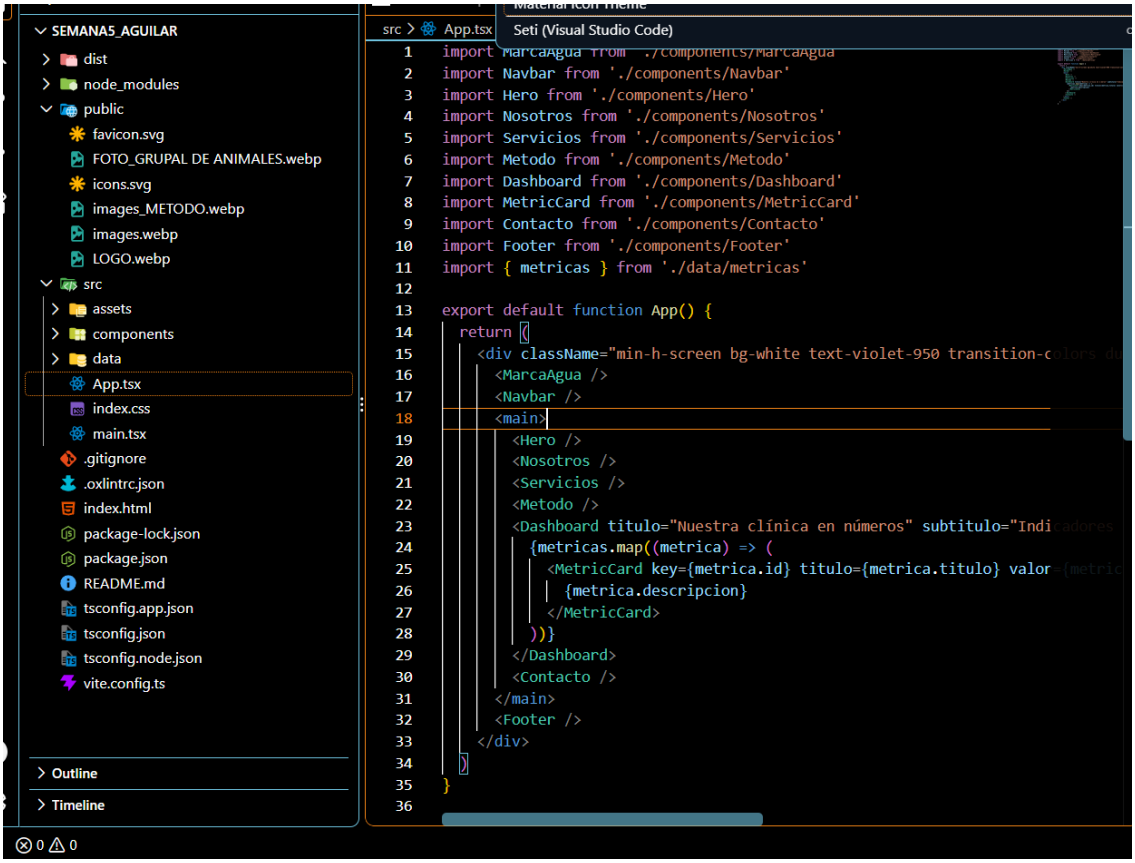




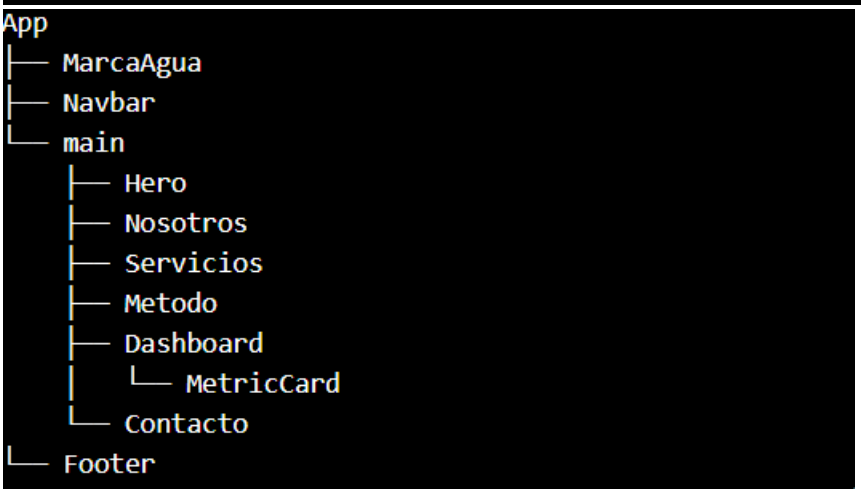




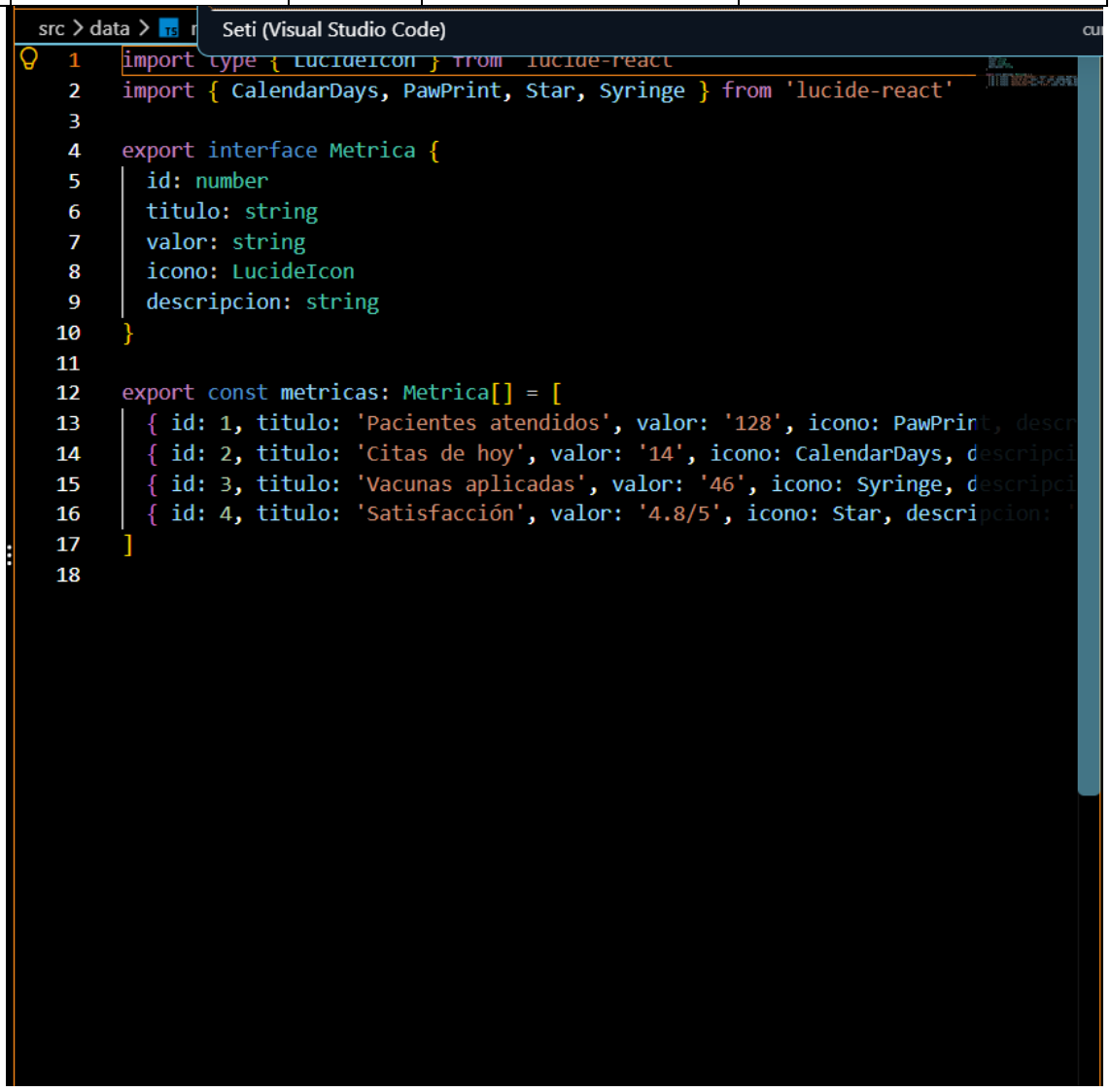
2	Limpiar boilerplate, crear src/components/, src/styles/, configurar alias o rutas relativas. Estructurar App.jsx como orquestador	8 min	Arquitectura de componentes, árbol de render, JSX → React.createElement	Mantener un solo componente raíz <App> que importa hijos. Evitar lógica de negocio en App


SEMANA5_AGUILAR
dist
node_modules
public
favicon.svg
FOTO_GRUPAL DE ANIMALES.webp
icons.svg
images_METODO.webp
images.webp
LOGO.webp
src
assets
components
data
App.tsx
index.css
main.tsx
.gitignore
.oxlintrc.json
index.html
package-lock.json
package.json
README.md
tsconfig.app.json
tsconfig.json
tsconfig.node.json
vite.config.ts
Outline
Timeline
src > App.tsx
Seti (Visual Studio Code)
import MarcaAgua from './components/MarcaAgua'
import Navbar from './components/Navbar'
import Hero from './components/Hero'
import Nosotros from './components/Nosotros'
import Servicios from './components/Servicios'
import Metodo from './components/Metodo'
import Dashboard from './components/Dashboard'
import MetricCard from './components/MetricCard'
import Contacto from './components/Contacto'
import Footer from './components/Footer'
import { metricas } from './data/metricas'
export default function App() {
return (
<div className="min-h-screen bg-white text-violet-950 transition-c
<MarcaAgua />
<Navbar />
<main>
<Hero />
<Nosotros />
<Servicios />
<Metodo />
<Dashboard titulo="Nuestra clínica en números" subtitulo="Indi
{metricas.map((metrica) => (
<MetricCard key={metrica.id} titulo={metrica.titulo} valor
{metrica.descripcion}
</MetricCard>
))}
</Dashboard>
<Contacto />
</main>
<Footer />

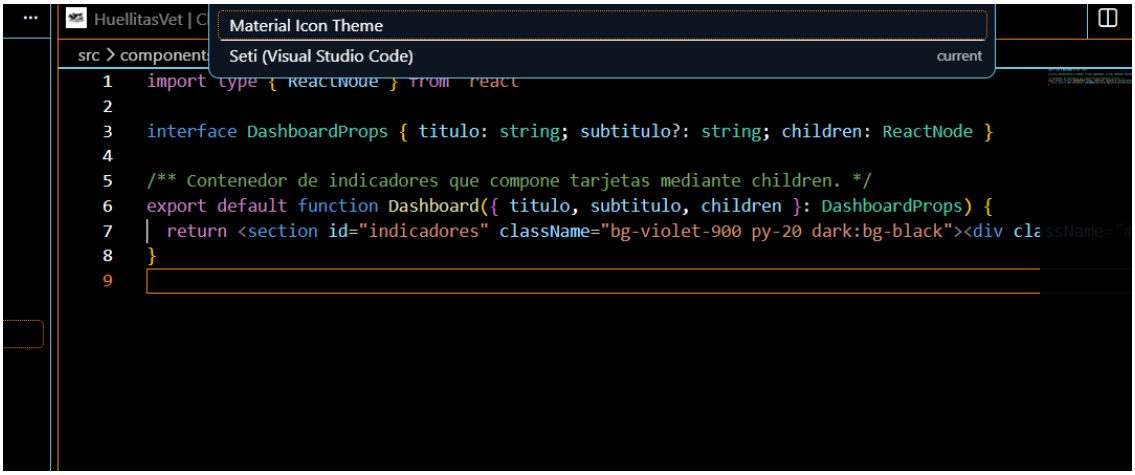
)
}
0 0

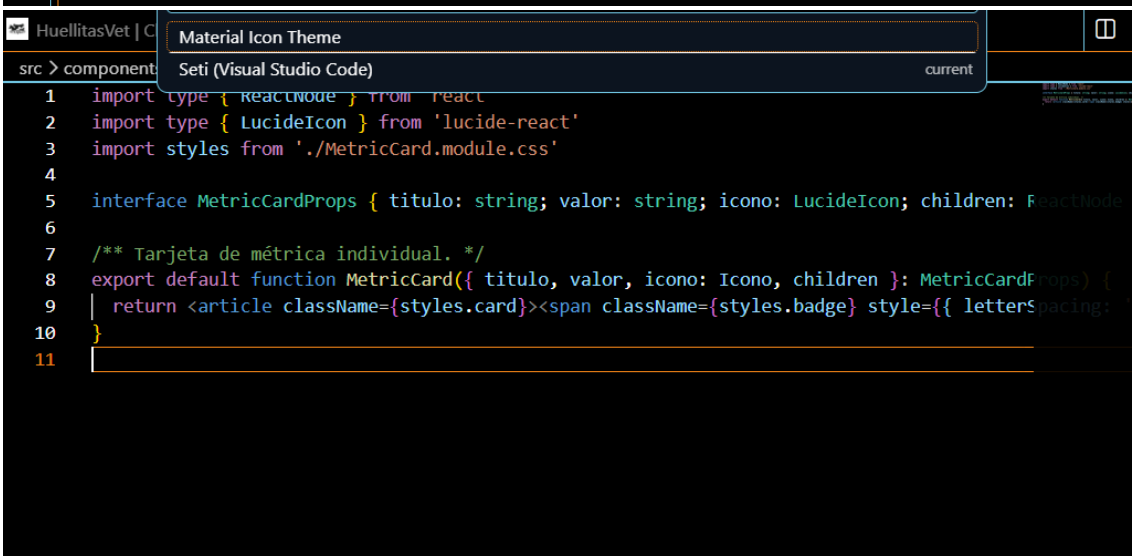

App
MarcaAgua
Navbar
main
Hero
Nosotros
Servicios
Metodo
Dashboard
MetricCard
Contacto
Footer

3	Implementar patrón props y children: crear <Dashboard> y <MetricCard>. Pasar datos estructurados y contenido anidado	12 min	Flujo unidireccional, inmutabilidad de props, composición vs herencia, PropTypes/JSDoc	Documentar tipos con JSDoc /** @param {Object} props */. No usar state en este paso



UNIVERSIDAD NACIONAL DEL CENTRO DEL PERÚ
FACULTAD DE INGENIERÍA DE SISTEMAS
DEPARTAMENTO ACADÉMICO DE INGENIERÍA DE SISTEMAS PROGRAMA DE INGENIERÍA DE SISTEMAS



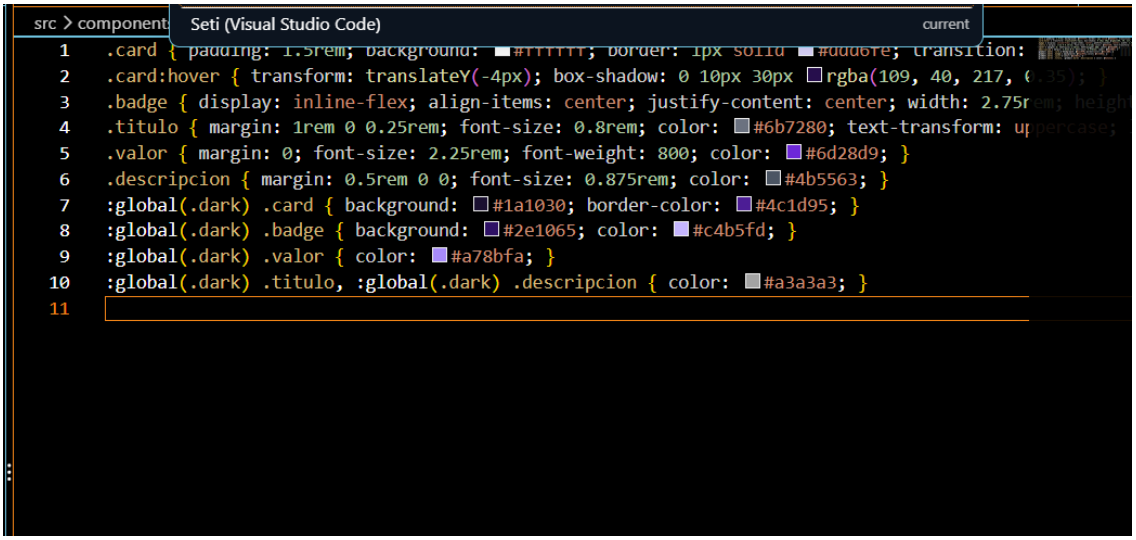


```jsx
<Dashboard
  titulo="Nuestra clínica en números"
  subtitulo="Indicadores de atención"
>
  {metricas.map((metrica) => (
    <MetricCard
      key={metrica.id}
      titulo={metrica.titulo}
      valor={metrica.valor}
      icono={metrica.icono}
    >
      {metrica.descripcion}
    </MetricCard>
  ))}
</Dashboard>
```

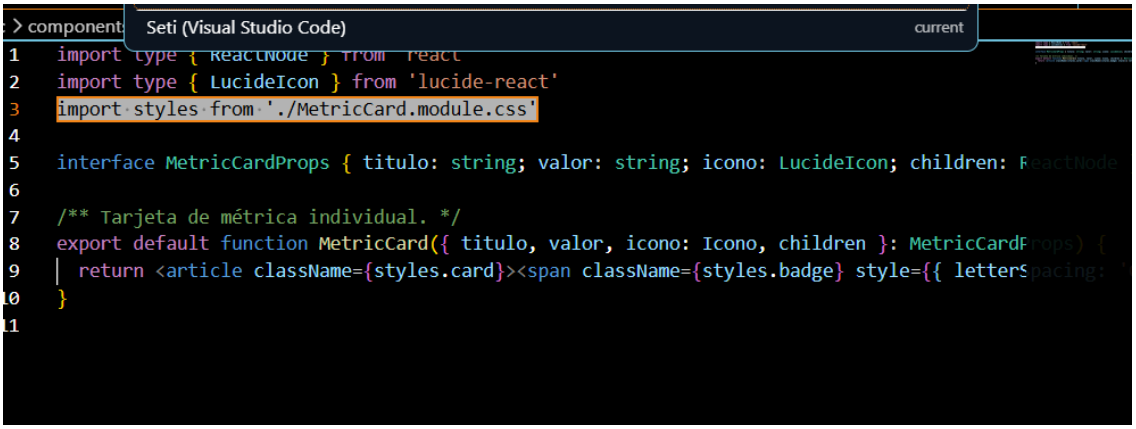
Conceptos que demuestra

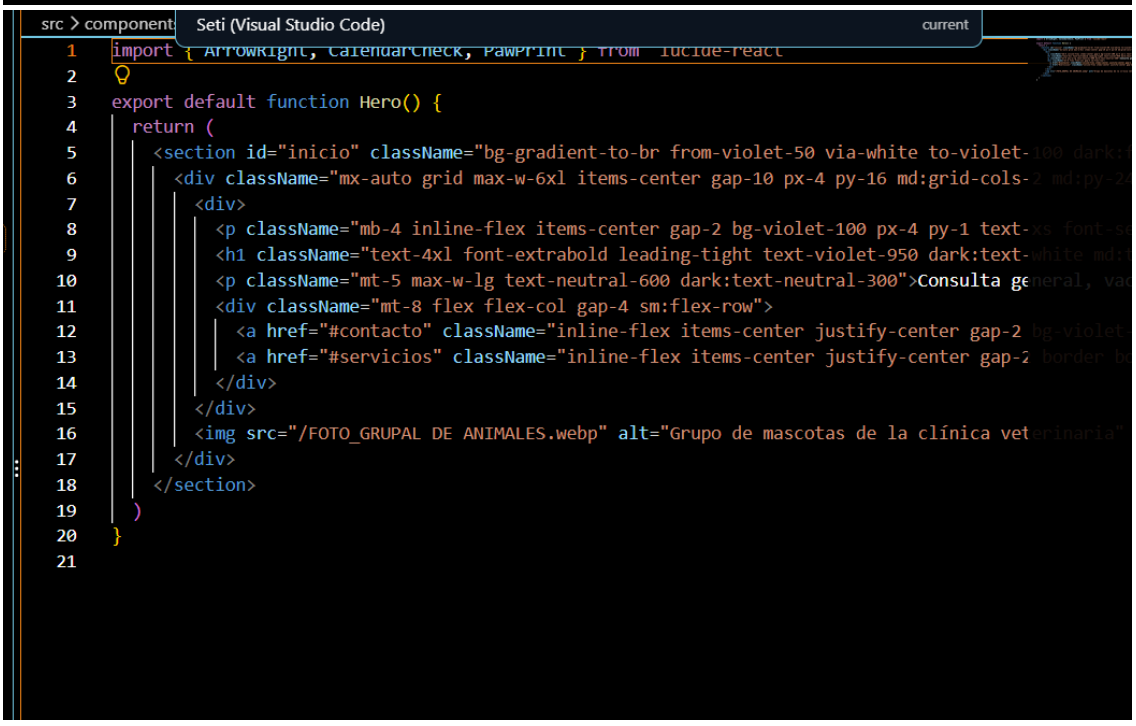
- `Dashboard` recibe:
  - `titulo`
  - `subtitulo`
  - `children`
- `MetricCard` recibe:
  - `titulo`
  - `valor`
  - `icono`
  - `children`

4	Aplicar estrategias de estilado: CSS Modules para <MetricCard>, utilidades Tailwind (si se configura) o inline para contrastar scope y colisiones	10 min	Scope de estilos, className vs class, CSS Modules hashing, prioridad de cascada en CSR	Prohibido usar <style> global en componentes. Usar *.module.css explícito



MANERA DE IMPORTACION





5	Validar CSR: inspeccionar HTML inicial (View Page Source), verificar hidratación en React DevTools, analizar red (JS	8 min	CSR vs SSR, hidratación, bundle analysis, index.html vacío vs DOM inyectado	Explicar por qué meta viewport y SEO tags deben ir en index.html, no en JSX
	chunks) y ciclo de render			

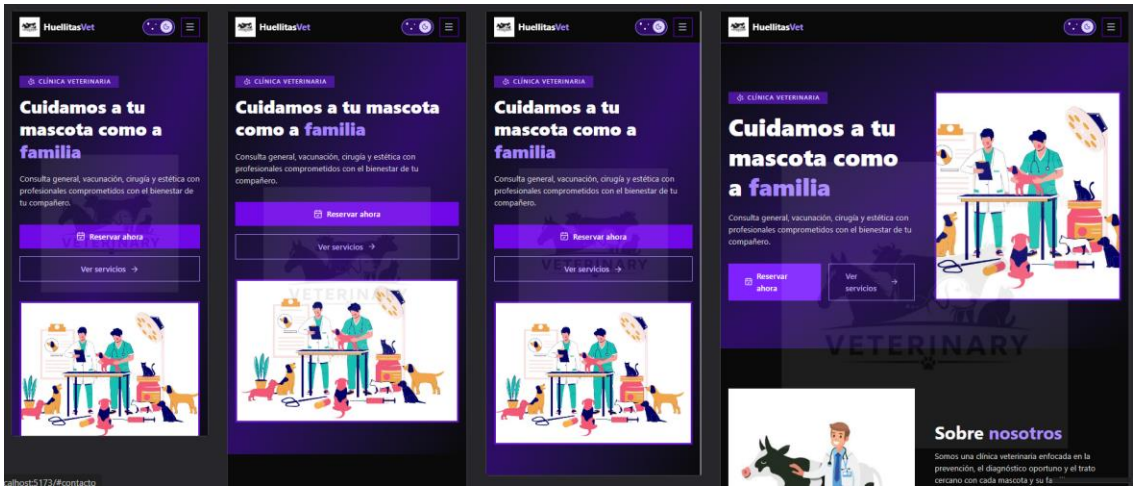


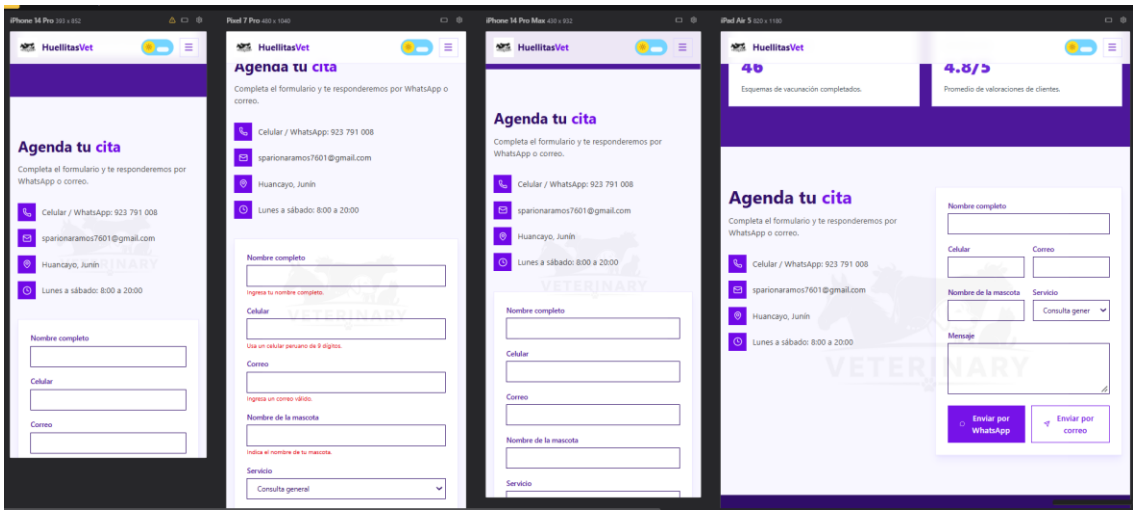
main.tsx — Seti (Visual Studio Code) — current

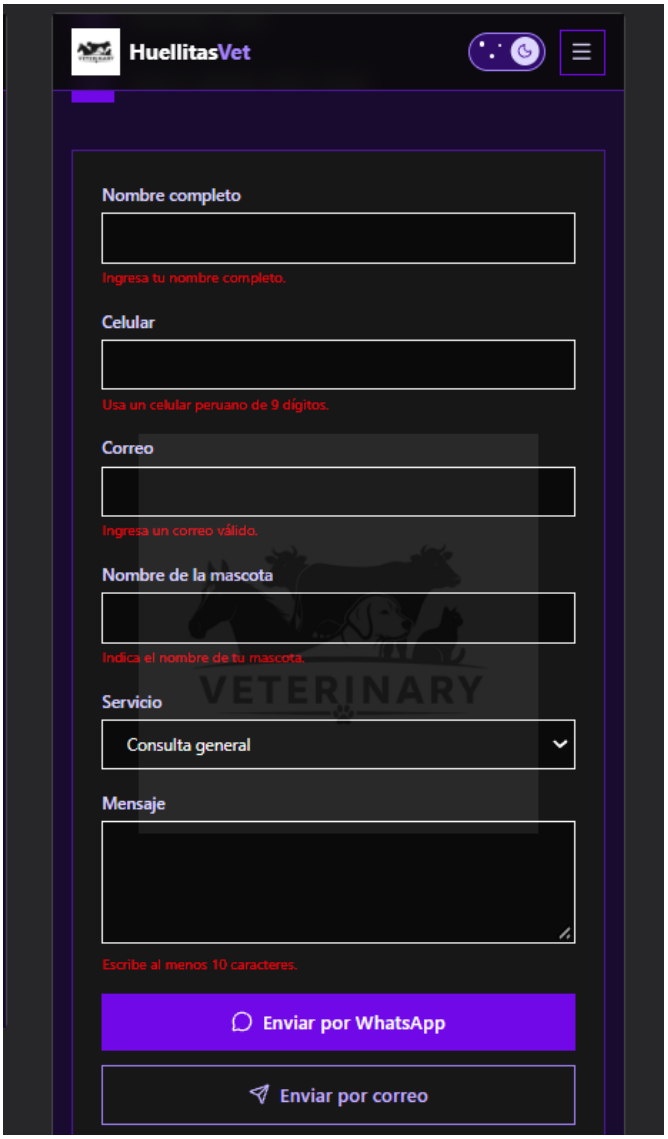
```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
import App from './App.tsx'

createRoot(document.getElementById('root')!).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

- Añada capturas de la Página Web implementada de manera local







- Proporcione el enlace de Github de su proyecto
  https://github.com/saulPariona/AGUILAR_SEMANA_5.git